

Odette

Generated by Doxygen 1.9.1

1 Namespace Index	1
1.1 Namespace List	1
2 File Index	3
2.1 File List	3
3 Namespace Documentation	5
3.1 barlog Namespace Reference	5
3.1.1 Function Documentation	5
3.1.1.1 error()	5
3.1.1.2 get_date_now()	6
3.1.1.3 info()	6
3.1.1.4 initialize_log()	6
3.1.1.5 warn()	6
3.1.1.6 write()	7
3.2 interpret Namespace Reference	7
3.2.1 Function Documentation	7
3.2.1.1 gauss_solve()	7
3.2.1.2 parse_tdm()	8
3.2.1.3 parse_tdm_w()	9
3.2.1.4 radec_to_unit_vector()	10
3.3 orbmath Namespace Reference	10
3.3.1 Function Documentation	10
3.3.1.1 arcsec2rad()	11
3.3.1.2 clamp()	11
3.3.1.3 compute_orbital_elements()	11
3.3.1.4 deg2rad()	12
3.3.1.5 geodetic_to_ecef()	12
3.3.1.6 kepler()	13
3.3.1.7 wrap_2pi()	13
3.4 propagate Namespace Reference	14
4 File Documentation	15
4.1 atmosphere.h File Reference	15
4.2 frame.h File Reference	15
4.3 iers_coef.h File Reference	15
4.4 interpret.cpp File Reference	15
4.5 interpret.h File Reference	16
4.6 logging.cpp File Reference	16
4.7 logging.h File Reference	17
4.8 lunar_position.h File Reference	18
4.9 orbmath.cpp File Reference	18
4.10 orbmath.h File Reference	19

4.11 orbmath_nutation.h File Reference	19
4.12 orbmath_perturbation.h File Reference	19
4.13 propagator.cpp File Reference	19
4.14 propagator.h File Reference	20
4.15 RADec.cpp File Reference	20
4.15.1 Macro Definition Documentation	21
4.15.1.1 frac	21
4.16 solar_position.h File Reference	21
4.17 TDMParser.cpp File Reference	21
4.18 TDMParser.h File Reference	22
Index	23

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

barlog	5
interpret	7
orbmath	10
propagate	14

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

atmosphere.h	15
frame.h	15
iers_coef.h	15
interpret.cpp	15
interpret.h	16
logging.cpp	16
logging.h	17
lunar_position.h	18
orbmath.cpp	18
orbmath.h	19
orbmath_nutation.h	19
orbmath_perturbation.h	19
propagator.cpp	19
propagator.h	20
RADec.cpp	20
solar_position.h	21
TDMParser.cpp	21
TDMParser.h	22

Chapter 3

Namespace Documentation

3.1 barlog Namespace Reference

Functions

- `std::ofstream write ("default_log.log")`
Output file stream used for logging messages.
- `std::string get_date_now ()`
Retrieve the current date and time as a formatted string.
- `void initialize_log ()`
Initialize the log file with a timestamped filename.
- `void info (const std::string &msg)`
Log an informational message.
- `void warn (const std::string &msg)`
Log a warning message.
- `void error (const std::string &msg)`
Log an error message.

3.1.1 Function Documentation

3.1.1.1 error()

```
void barlog::error (  
    const std::string & msg )
```

Log an error message.

Prepends the current date-time and "[ERROR]" level tag before writing the provided message to the log file.

Parameters

<i>msg</i>	The error message to log.
------------	---------------------------

3.1.1.2 get_date_now()

```
std::string barlog::get_date_now ( )
```

Retrieve the current date and time as a formatted string.

Formats the current local time into the pattern "DD-MM-YYYY-HH-MM-SS".

Returns

std::string The formatted date-time string.

3.1.1.3 info()

```
void barlog::info (
    const std::string & msg )
```

Log an informational message.

Prepends the current date-time and "[INFO]" level tag before writing the provided message to the log file.

Parameters

<i>msg</i>	The informational message to log.
------------	-----------------------------------

3.1.1.4 initialize_log()

```
void barlog::initialize_log ( )
```

Initialize the log file with a timestamped filename.

Closes the default log file (if open) and opens a new file named "<current_date_time>.log" to capture subsequent log entries.

3.1.1.5 warn()

```
void barlog::warn (
    const std::string & msg )
```

Log a warning message.

Prepends the current date-time and "[WARN]" level tag before writing the provided message to the log file.

Parameters

<code>msg</code>	The warning message to log.
------------------	-----------------------------

3.1.1.6 write()

```
std::ofstream barlog::write (
    "default_log.log" )
```

Output file stream used for logging messages.

By default, logs are written to "default_log.log" until [initialize_log\(\)](#) is called to create a timestamped log file.

3.2 interpret Namespace Reference**Functions**

- Eigen::Vector3d [radec_to_unit_vector](#) (double ra, double dec)
Convert right ascension and declination angles into a unit direction vector.
- StateVectors [gauss_solve](#) (std::vector< Observation > &observations)
Solve for initial position and velocity vectors using Gauss's method.
- TDMDData [parse_tdm](#) (const std::string &filename)
Parse a single TDM (Tracking Data Message) file and extract observations.
- TDMDData [parse_tdm_w](#) (const std::string &wildcard, double delta, double delta_error)
Parse multiple TDM files matching a wildcard pattern and filter observations by time interval.

3.2.1 Function Documentation**3.2.1.1 gauss_solve()**

```
StateVectors interpret::gauss_solve (
    std::vector< Observation > & observations )
```

Solve for initial position and velocity vectors using Gauss's method.

This function takes at least three optical observations (each containing an epoch, station ECEF position, and right ascension/declination angles) and computes the approximate satellite state vectors (position and velocity) at the middle observation's epoch. It uses the classical Gauss method of orbit determination:

1. Converts observed RA/Dec to line-of-sight unit vectors.
2. Transforms station ECEF coordinates to ECI at each observation epoch.
3. Constructs the Lambert-like linear system to estimate slant ranges.
4. Solves a ninth-degree polynomial for the middle slant range magnitude.
5. Computes the position vectors r1, r2, r3 in ECI coordinates.
6. Uses Gibbs method to estimate the velocity vector at r2.

Parameters

<i>observations</i>	A reference to a vector of at least three Observation structs. Each Observation must contain: • epoch : Time in days since J2000 (UTC). • stationECEF: Station position in ECEF coordinates (kilometers). • ra, dec : Right ascension and declination in radians.
---------------------	---

Returns

StateVectors A struct containing:

- epoch: The epoch corresponding to the second observation.
- r : Position vector at the second observation's epoch (kilometers, ECI).
- v : Velocity vector at the second observation's epoch (km/s, ECI).

Exceptions

<i>std::invalid_argument</i>	If fewer than three observations are provided.
<i>std::runtime_error</i>	If no valid positive real root is found for the slant-range polynomial.

Note

All internal angle variables are assumed to be in radians. The gravitational parameter mu (μ) is assumed to be defined in orbmath namespace (km^3/s^2).

3.2.1.2 parse_tdm()

```
TDMDData interpret::parse_tdm (
    const std::string & filename )
```

Parse a single TDM (Tracking Data Message) file and extract observations.

Opens the specified TDM file, reads the metadata between META_START and META_STOP markers, and reads observation lines between DATA_START and DATA_STOP. Metadata fields such as TIME_SYSTEM, START_TIME, STOP_TIME, and REFERENCE_FRAME are stored in the returned TDMDData.meta structure. Station geodetic coordinates (longitude, latitude, altitude) are parsed from COMMENT lines and converted to an ECEF position. Observation lines consist of timestamp, right ascension (degrees), and declination (degrees). Each pair of lines yields one Observation entry with epoch (Julian date), ra (radians), dec (radians), and stationECEF (ECEF position in kilometers). All observations are stored in TDMDData.observations.

Parameters

<i>filename</i>	Filesystem path to the TDM file to parse.
-----------------	---

Returns

TDMDData A struct containing parsed metadata and a vector of Observation entries.

Exceptions

<code>std::runtime_error</code>	If the file cannot be opened.
---------------------------------	-------------------------------

Note

Logs start and end of parsing via [barlog::info](#), and errors via [barlog::error](#).

3.2.1.3 parse_tdm_w()

```
TDMDData interpret::parse_tdm_w (
    const std::string & wildcard,
    double delta,
    double delta_error )
```

Parse multiple TDM files matching a wildcard pattern and filter observations by time interval.

Given a filesystem wildcard (e.g., "/path/to/dir/*.tdm"), this function:

1. Extracts the directory and filename pattern from the wildcard string.
2. Iterates over all regular files in that directory whose names contain the pattern prefix.
3. Invokes [parse_tdm\(\)](#) on each matching file to collect TDMDData objects.
4. Merges all observations into a single TDMDData, sorting them by epoch.
5. Filters the merged observations to include only those where the time difference from the previous selected observation is within [delta - delta_error, delta + delta_error] seconds.
6. Returns the filtered, merged TDMDData with metadata inherited from the first file.

Parameters

<i>wildcard</i>	A wildcard string specifying directory and filename pattern (e.g., "/path/to/*.tdm").
<i>delta</i>	Desired time separation between observations, in seconds.
<i>delta_error</i>	Allowed tolerance for time separation, in seconds.

Returns

TDMDData A struct containing the merged metadata and filtered observations.

Exceptions

<code>std::runtime_error</code>	If the directory portion does not exist or is not a directory, or if no files match the wildcard.
---------------------------------	---

Note

Logs warnings if delta or delta_error are non-positive, and logs errors for I/O issues.

3.2.1.4 radec_to_unit_vector()

```
Eigen::Vector3d interpret::radec_to_unit_vector (
    double ra,
    double dec )
```

Convert right ascension and declination angles into a unit direction vector.

Given right ascension (ra) and declination (dec) in radians, this function computes the corresponding unit vector in Cartesian coordinates: $x = \cos(\text{dec}) * \cos(\text{ra})$ $y = \cos(\text{dec}) * \sin(\text{ra})$ $z = \sin(\text{dec})$

Parameters

<i>ra</i>	Right ascension angle, in radians.
<i>dec</i>	Declination angle, in radians.

Returns

Eigen::Vector3d A unit vector pointing in the direction specified by (ra, dec).

3.3 orbmath Namespace Reference**Functions**

- double [deg2rad](#) (double deg)
Convert degrees to radians.
- double [arcsec2rad](#) (double arcsec)
Convert arcseconds to radians.
- double [wrap_2pi](#) (double x)
Wrap an angle to the range [0, 2pi).
- double [clamp](#) (double x, double low=-1.0, double high=1.0)
Clamp a value to a specified range.
- Eigen::Vector3d [geodetic_to_ecef](#) (double latDeg, double lonDeg, double alt_m)
Convert geodetic coordinates to Earth-Centered Earth-Fixed (ECEF).
- OrbitalElements [compute_orbital_elements](#) (const Eigen::Vector3d &r, const Eigen::Vector3d &v)
Compute classical orbital elements from position and velocity vectors.
- Eigen::Vector3d [kepler](#) (const Eigen::Vector3d &r, double dt)
Compute the two-body (Keplerian) acceleration.

3.3.1 Function Documentation

3.3.1.1 arcsec2rad()

```
double orbmath::arcsec2rad (
    double arcsec )
```

Convert arcseconds to radians.

Given an angle in arcseconds, returns the equivalent angle in radians. Note: 1 arcsecond = 1/3600 degree.

Parameters

<i>arcsec</i>	Angle in arcseconds.
---------------	----------------------

Returns

double Angle in radians.

3.3.1.2 clamp()

```
double orbmath::clamp (
    double x,
    double low = -1.0,
    double high = 1.0 )
```

Clamp a value to a specified range.

Ensures that x lies within the closed interval $[low, high]$. This is particularly useful for guarding against domain errors in functions like $\arccos$.

Parameters

x	Value to clamp.
<i>low</i>	Lower bound (default: -1.0).
<i>high</i>	Upper bound (default: 1.0).

Returns

double Clamped value in $[low, high]$.

3.3.1.3 compute_orbital_elements()

```
OrbitalElements orbmath::compute_orbital_elements (
    const Eigen::Vector3d & r,
    const Eigen::Vector3d & v )
```

Compute classical orbital elements from position and velocity vectors.

Given a position vector r and velocity vector v in an inertial frame (kilometers and kilometers per second), computes the six classical orbital elements:

- Semi-major axis (*a*)
- Eccentricity (*ecc*)
- Inclination (*incl*)
- Argument of perigee (*omega*)
- Right ascension of ascending node (*Omega*)
- True anomaly (*nu*)
- Mean anomaly (*m*)

The angles in the returned `OrbitalElements` structure are expressed in degrees.

Parameters

<i>r</i>	Position vector [x, y, z] in kilometers.
<i>v</i>	Velocity vector [vx, vy, vz] in kilometers per second.

Returns

`OrbitalElements` Struct containing the computed orbital elements.

3.3.1.4 `deg2rad()`

```
double orbmath::deg2rad (
    double deg )
```

Convert degrees to radians.

Given an angle in degrees, returns the equivalent angle in radians.

Parameters

<i>deg</i>	Angle in degrees.
------------	-------------------

Returns

double Angle in radians.

3.3.1.5 `geodetic_to_ecef()`

```
Eigen::Vector3d orbmath::geodetic_to_ecef (
    double latDeg,
    double lonDeg,
    double alt_m )
```


Convert geodetic coordinates to Earth-Centered Earth-Fixed (ECEF).

Uses the WGS-84 ellipsoid model to convert latitude, longitude, and altitude into an ECEF position vector in kilometers.

Parameters

<i>latDeg</i>	Geodetic latitude in degrees.
<i>lonDeg</i>	Geodetic longitude in degrees.
<i>alt_m</i>	Altitude above the WGS-84 ellipsoid in meters.

Returns

Eigen::Vector3d Position vector [x, y, z] in kilometers (ECEF).

3.3.1.6 kepler()

```
Eigen::Vector3d orbmath::kepler (
    const Eigen::Vector3d & r,
    double dt )
```

Compute the two-body (Keplerian) acceleration.

Given the position vector *r* (in kilometers) and a time step *dt* (unused in the simple central-force model), returns the acceleration vector due to the Earth's gravitational parameter (*mu*).

Parameters

<i>r</i>	Position vector [x, y, z] in kilometers.
<i>dt</i>	Time step in seconds (not used for constant-force evaluation).

Returns

Eigen::Vector3d Acceleration vector [ax, ay, az] in km/s².

3.3.1.7 wrap_2pi()

```
double orbmath::wrap_2pi (
    double x )
```

Wrap an angle to the range [0, 2pi).

Reduces any real-valued angle *x* to the principal value in the range [0, 2pi).

Parameters

x	Angle in radians (may be outside $[0, 2\pi)$ interval).
-----	---

Returns

double Wrapped angle in $[0, 2\pi)$.

3.4 propagate Namespace Reference

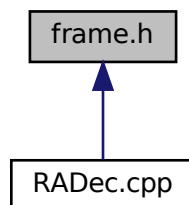
Chapter 4

File Documentation

4.1 atmosphere.h File Reference

4.2 frame.h File Reference

This graph shows which files directly or indirectly include this file:



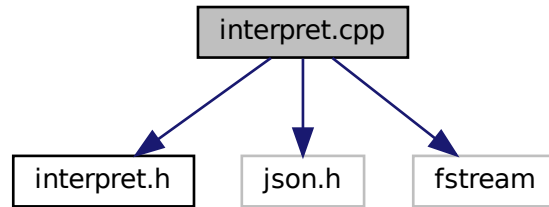
4.3 iers_coef.h File Reference

4.4 interpret.cpp File Reference

```
#include "interpret.h"  
#include "json.h"
```

```
#include <fstream>
```

Include dependency graph for `interpret.cpp`:

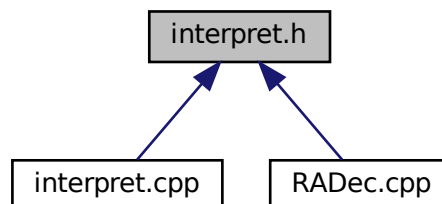


Namespaces

- [interpret](#)

4.5 `interpret.h` File Reference

This graph shows which files directly or indirectly include this file:

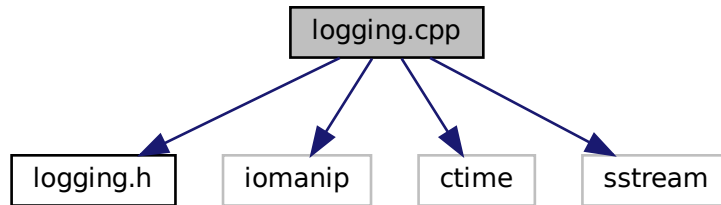


4.6 `logging.cpp` File Reference

```
#include "logging.h"  
#include <iomanip>  
#include <ctime>
```

```
#include <sstream>
```

Include dependency graph for logging.cpp:



Namespaces

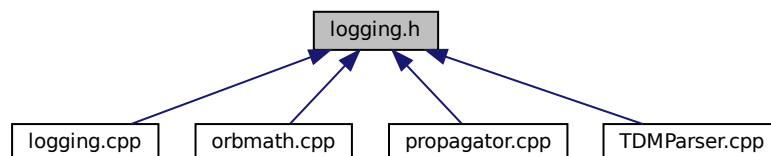
- [barlog](#)

Functions

- `std::ofstream barlog::write ("default_log.log")`
Output file stream used for logging messages.
- `std::string barlog::get\_date\_now ()`
Retrieve the current date and time as a formatted string.
- `void barlog::initialize\_log ()`
Initialize the log file with a timestamped filename.
- `void barlog::info (const std::string &msg)`
Log an informational message.
- `void barlog::warn (const std::string &msg)`
Log a warning message.
- `void barlog::error (const std::string &msg)`
Log an error message.

4.7 logging.h File Reference

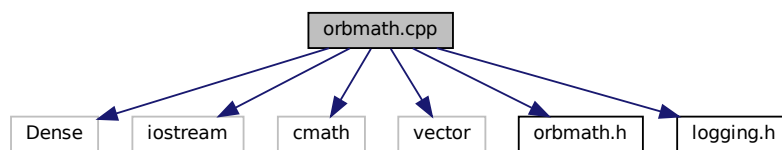
This graph shows which files directly or indirectly include this file:



4.8 lunar_position.h File Reference

4.9 orbmath.cpp File Reference

```
#include <Dense>
#include <iostream>
#include <cmath>
#include <vector>
#include "orbmath.h"
#include "logging.h"
Include dependency graph for orbmath.cpp:
```



Namespaces

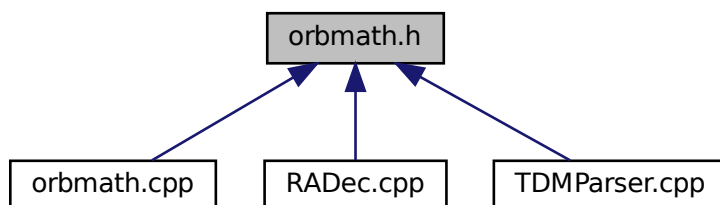
- [orbmath](#)

Functions

- double [orbmath::deg2rad](#) (double deg)
Convert degrees to radians.
- double [orbmath::arcsec2rad](#) (double arcsec)
Convert arcseconds to radians.
- double [orbmath::wrap_2pi](#) (double x)
Wrap an angle to the range $[0, 2\pi)$.
- double [orbmath::clamp](#) (double x, double low=-1.0, double high=1.0)
Clamp a value to a specified range.
- Eigen::Vector3d [orbmath::geodetic_to_ecef](#) (double latDeg, double lonDeg, double alt_m)
Convert geodetic coordinates to Earth-Centered Earth-Fixed (ECEF).
- OrbitalElements [orbmath::compute_orbital_elements](#) (const Eigen::Vector3d &r, const Eigen::Vector3d &v)
Compute classical orbital elements from position and velocity vectors.
- Eigen::Vector3d [orbmath::kepler](#) (const Eigen::Vector3d &r, double dt)
Compute the two-body (Keplerian) acceleration.

4.10 orbmath.h File Reference

This graph shows which files directly or indirectly include this file:



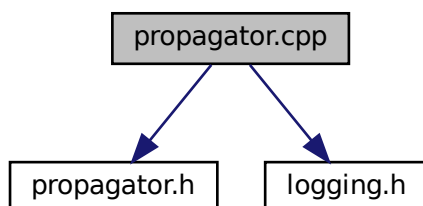
4.11 orbmath_nutation.h File Reference

4.12 orbmath_perturbation.h File Reference

4.13 propagator.cpp File Reference

```
#include "propagator.h"  
#include "logging.h"
```

Include dependency graph for `propagator.cpp`:

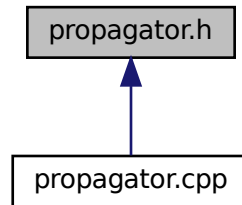


Namespaces

- [propagate](#)

4.14 propagator.h File Reference

This graph shows which files directly or indirectly include this file:



4.15 RADec.cpp File Reference

```

#include "interpret.h"
#include <iostream>
#include <Dense>
#include <cmath>
#include <format>
#include <stdexcept>
#include <vector>
#include <limits>
#include "TDMParser.h"
#include <iomanip>
#include "orbmath.h"
#include <unsupported/Eigen/Polynomials>
#include "frame.h"

```

Include dependency graph for RADec.cpp:



Namespaces

- [interpret](#)

Macros

- `#define` [frac\(x\)](#) (`x-floor(x)`)

Functions

- Eigen::Vector3d [interpret::radec_to_unit_vector](#) (double ra, double dec)
Convert right ascension and declination angles into a unit direction vector.
- StateVectors [interpret::gauss_solve](#) (std::vector< Observation > &observations)
Solve for initial position and velocity vectors using Gauss's method.

4.15.1 Macro Definition Documentation

4.15.1.1 frac

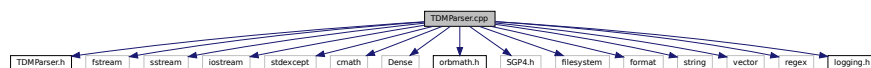
```
#define frac(  
    x ) (x-floor(x))
```

4.16 solar_position.h File Reference

4.17 TDMParser.cpp File Reference

```
#include "TDMParser.h"  
#include <fstream>  
#include <sstream>  
#include <iostream>  
#include <stdexcept>  
#include <cmath>  
#include <Dense>  
#include "orbmath.h"  
#include "SGP4.h"  
#include <filesystem>  
#include <format>  
#include <string>  
#include <vector>  
#include <regex>  
#include "logging.h"
```

Include dependency graph for TDMParser.cpp:



Namespaces

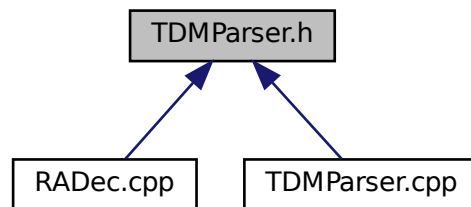
- [interpret](#)

Functions

- TDMDData [interpret::parse_tdm](#) (const std::string &filename)
Parse a single TDM (Tracking Data Message) file and extract observations.
- TDMDData [interpret::parse_tdm_w](#) (const std::string &wildcard, double delta, double delta_error)
Parse multiple TDM files matching a wildcard pattern and filter observations by time interval.

4.18 TDMParser.h File Reference

This graph shows which files directly or indirectly include this file:



Index

- arcsec2rad
 - orbmath, 10
- atmosphere.h, 15
- barlog, 5
 - error, 5
 - get_date_now, 6
 - info, 6
 - initialize_log, 6
 - warn, 6
 - write, 7
- clamp
 - orbmath, 11
- compute_orbital_elements
 - orbmath, 11
- deg2rad
 - orbmath, 12
- error
 - barlog, 5
- frac
 - RADec.cpp, 21
- frame.h, 15
- gauss_solve
 - interpret, 7
- geodetic_to_ecef
 - orbmath, 12
- get_date_now
 - barlog, 6
- iers_coef.h, 15
- info
 - barlog, 6
- initialize_log
 - barlog, 6
- interpret, 7
 - gauss_solve, 7
 - parse_tdm, 8
 - parse_tdm_w, 9
 - radec_to_unit_vector, 10
- interpret.cpp, 15
- interpret.h, 16
- kepler
 - orbmath, 13
- logging.cpp, 16
- logging.h, 17
- lunar_position.h, 18
- orbmath, 10
 - arcsec2rad, 10
 - clamp, 11
 - compute_orbital_elements, 11
 - deg2rad, 12
 - geodetic_to_ecef, 12
 - kepler, 13
 - wrap_2pi, 13
- orbmath.cpp, 18
- orbmath.h, 19
- orbmath_nutation.h, 19
- orbmath_perturbation.h, 19
- parse_tdm
 - interpret, 8
- parse_tdm_w
 - interpret, 9
- propagate, 14
- propagator.cpp, 19
- propagator.h, 20
- RADec.cpp, 20
 - frac, 21
- radec_to_unit_vector
 - interpret, 10
- solar_position.h, 21
- TDMParser.cpp, 21
- TDMParser.h, 22
- warn
 - barlog, 6
- wrap_2pi
 - orbmath, 13
- write
 - barlog, 7